



Sinhgad Technical Education Society's  
**SINHGAD INSTITUTE OF TECHNOLOGY & SCIENCE,**  
Narhe ,Pune- 411041

**Department of Electronics &  
Telecommunication Engineering**

**Lab Manual**

**Network Security(EL-II)**

TE E&TC (2019 Course)

University Seat Number:



**Sinhgad Technical Education Society's**  
**SINHGAD INSTITUTE OF TECHNOLOGY & SCIENCE,**  
**Department of Electronics & Telecommunication Engineering**

**CERTIFICATE**

*This is certifying that,*

*Mr. / Miss.....*

*Student of ..... Year.....Engineering for  
the academic year 20 - 20 has carried out all the tutorials in the  
index to our satisfaction.*

*He / She have performed ..... Tutorials out of .....*

*Tutorials of the Subject .....*

*Date: - .....*

*Subject Incharge*

*Head of the Department*



**Sinhgad Institutes**

[illegible]



## SEMESTER - II

CLASS: - T.E [ETC]      SUBJECT: - EL-II-NS      DATE:      /      / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 1

**\*\* Design and implement for the insecurity of default passwords, printed passwords and password transmitted in plain text. \*\***

**AIM :** Write a program to Design and implement for the insecurity of default passwords, printed passwords and password transmitted in plain text.

#### **OBJECTIVE:**

1. To understand the concept of insecurity of default passwords, printed passwords and password transmitted in plain text.

#### **THEORY:**

A password, sometimes called a passcode is secret data, typically a string of characters, usually used to confirm a user's identity. In general, a password is an arbitrary string of characters including letters, digits, or other symbols. If the permissible characters are constrained to be numeric, the corresponding secret is sometimes called a personal identification number (PIN).

##### **1. Default passwords:**

A default password is a standard pre-configured password for a device. Such passwords are the default configuration for many devices and, if unchanged, present a serious security risk. Typical examples of default passwords include admin, password and guest. Furthermore, a vendor generally uses a single default password, which can be easily found online through search or on websites that provide compiled lists.

Default passwords are commonly used for routers, access points, switches and firewalls. They are also common in embedded systems, industrial control systems (ICS) and remote terminal interfaces such as Telnet and SSH.

Left unchanged, default passwords provide an easy attack vector for home network equipment; if the owner also connects to a corporate network that risk extends to the business as well. An attacker who logs into a device successfully is likely to have administrative-level access.

The risk is also severe in the case of embedded systems and ICS security because these environments were not originally intended to be accessible over the internet. However, in the burgeoning internet of things (IoT) environment, almost anything can be made addressable and connected, and while there are many benefits to IoT connectivity, enhanced security is not among them.

Default passwords are intended to be place holders and used only for the initial setup of hardware or after a factory reset. The user enters the password and is usually prompted to change it as part of the process, but not always.

To counteract the security issues of default passwords, device owners should change the default value to a strong password when they first configure a device. Vendors can enforce that change upon first use of the default password. Another option for the vendor is to employ secure unique default passwords.

## **2. Printed Passwords:**

Most applications do not display the real password in the login box for security reasons, and instead, they show the asterisks (\*\*\*\*\*). But often, there is a need to know the actual password, especially if you have forgotten the password that you entered a while ago.

## **3. Plain Text Password:**

If the password is transmitted from the user to the server as plaintext (what you see is exactly what you get; it isn't hidden in any way) – it could be intercepted as it travels across the network.

This is usually overcome by encrypting the communication between the user and the server. The most common form of encryption is the Transport Layer Security (TLS) standard or the older SSL standard (Secure Socket Layer). You'll recognise that TLS or SSL is being used when you see 'https'

at the beginning of a web page address instead of 'http', and by a padlock symbol in your browser. (You'll look at encryption and TLS and SSL more fully in Week 4.)

Another problem occurs if a password is stored on a server as plaintext. In this case a successful attack on the server would not only reveal the user's password, but all the passwords for all the users of the system. However, when a user enters a password the server needs to be able to confirm that this is the correct password for that user before it grants access.

This second problem can also be solved using a technique called hashing. A hash with salt is the result of processing plaintext to create a unique, fixed length identifier – you'll find out more in Week 5. It cannot be used to reconstruct the original data – even if the hash falls into hostile hands. In this scheme, a hashing function is used to create a hash of a password, which is stored on the server – the password itself is discarded. When the user enters a password, this is sent over the network and hashed on the server using a copy of the same hashing function. The resulting hash is compared to the hash stored on the password server. Only if they match will the user be granted access. Some implementations of this scheme will hash the user's password before sending it across the network to be compared with the hash stored on the server. Almost all online services and computer systems store passwords as hashes – but surprisingly, errors still happen.

To avoid insecurity of password use **Strong Password**.

A **Strong Password** is defined as a password that is reasonably difficult to guess in a short period of time either through human guessing or the use of specialized software. The following are general recommendations for creating a Strong Password:

A Strong Password should -

- Be at least 8 characters in length
- Contain both upper and lowercase alphabetic characters (e.g. A-Z, a-z)
- Have at least one numerical character (e.g. 0-9)
- Have at least one special character (e.g. ~!@#\$\$%^&\*()\_+=)

Strong Passwords do not -

- Spell a word or series of words that can be found in a standard dictionary
- Spell a word with a number added to the beginning and the end
- Be based on any personal information such as user id, family name, pet, birthday, etc.

### Code(C Language):

#### 1. Default Passwords:

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

int main()
{
    char username[15];
    char password[12];

    printf("Enter your username:\n");
    scanf("%s",&username);

    printf("Enter your password:\n");
    scanf("%s",&password);

    if(strcmp(username,"admin")==0)
    {
        if(strcmp(password,"admin")==0)
        {
            printf("\n Welcome.Login Success!");
        }
    }
}
```

```
else
{
    printf("\nwrong password");
}
else
{
    printf("\n User doesn't exist");
}

return 0;
}
```

## Output:

```
main.c:12:13: warning: format '%s' expects argument of type 'char *', but argument 2 has type 'char (*)[15]' [-Wformat]
12 |     scanf("%s", &username);
    |     ~^ ~~~~~
    |     | |
    |     | char (*)[15]
    |     char *
main.c:15:13: warning: format '%s' expects argument of type 'char *', but argument 2 has type 'char (*)[12]' [-Wformat]
15 |     scanf("%s", &password);
    |     ~^ ~~~~~
    |     | |
    |     | char (*)[12]
    |     char *
Enter your username:
admin
Enter your password:
admin

Welcome.Login Success!

...Program finished with exit code 0
Press ENTER to exit console.
```

## Code (turbo c):

## 2. Printed Passwords:

```
#include <stdio.h>
#include <conio.h>

void main()
{
    char pasword[10],username[10], ch;
    int i;
    clrscr();

    printf("Enter User name: ");
    gets(username);
    printf("Enter the password <any 8 characters>: ");

    for(i=0;i<8;i++)
    {
        ch = getch();
        pasword[i] = ch;
        ch = '*';
        printf("%c",ch);
    }

    pasword[i] = '*';
    getch();
    /*If you want to know what you have entered as password, you can print it*/
    printf("\n","Your password is :");
    getch();
    for(i=0;i<8;i++)
    {
        printf("%c",pasword[i]);
        getch();
    }

}
```

## Output:

```
File Edit Search Run Compile Debug Project Options Window Help
Output
Enter User name: sinhgad
Enter the password <any 8 characters>: *****
students
```

**Conclusion:**

---

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]    SUBJECT: - EL-II-NS    DATE:        /        / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 2

### **\*\* ENCRYPTION AND DECRYPTION \*\***

**AIM :** Write a program for Encryption and Decryption.

**OBJECTIVE:**

1. To understand the concept of encryption and decryption.
2. Write programs for encryption and decryption using Ceaser Cipher.

**THEORY:**

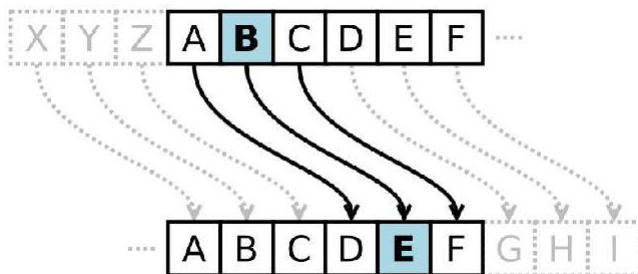
The Caesar Cipher technique is one of the earliest and simplest method of encryption technique. It's simply a type of substitution cipher, i.e., each letter of a given text is replaced by a letter some fixed number of positions down the alphabet. For example with a shift of 1, A would be replaced by B, B would become C, and so on. The method is apparently named after Julius Caesar, who apparently used it to communicate with his officials. Thus to cipher a given text we need an integer value, known as shift which indicates the number of position each letter of the text has been moved down. The encryption can be represented using modular arithmetic by first transforming the letters into numbers, according to the scheme, A = 0, B = 1,..., Z = 25. Encryption of a letter by a shift n can be described mathematically as.

$$E_n(x) = (x + n) \bmod 26 \quad (\text{Encryption Phase with shift } n)$$

$$D_n(x) = (x - n) \bmod 26 \quad (\text{Decryption Phase with shift } n)$$



Here is a Caesar cipher using a right rotation of three places, equivalent to a left shift of 23 (the shift parameter is used as the key):



| Plain  | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z |
|--------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Cipher | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | A | B | C |

When encrypting, a person looks up each letter of the message in the "plain" line and writes down the corresponding letter in the "cipher" line.

### Code (C Language):

```
//Simple C program to encrypt and decrypt a string
#include <stdio.h>

int main()
{
    int i, x;
    char str[100];
    printf("\nPlease enter a string:\t");
```

```
gets(str);

printf("\nPlease choose following options:\n");

printf("1 = Encrypt the string.\n");

printf("2 = Decrypt the string.\n");

scanf("%d", &x);

//using switch case statements

switch(x)
{
    case 1:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] + 3; //the key for encryption is 3 that is added to ASCII value
        printf("\nEncrypted string: %s\n", str);
        break;
    case 2:
        for(i = 0; (i < 100 && str[i] != '\0'); i++)
            str[i] = str[i] - 3; //the key for encryption is 3 that is subtracted to ASCII value
        printf("\nDecrypted string: %s\n", str);
        break;
    default:
        printf("\nError\n");
}

return 0;

}
```

## Output:

```
File Edit Search Run Compile Debug Project Options Window Help
Output
Please choose following options:
1 = Encrypt the string.
2 = Decrypt the string.
1
Encrypted string: vlqkjdq
Please enter a string: vlqkjdq
Please choose following options:
1 = Encrypt the string.
2 = Decrypt the string.
2
Decrypted string: sinhgad
```

## Conclusion:

---

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]    SUBJECT: - EL-II-NS    DATE:        /        / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 3

#### **\*\* Monoalphabetic Cipher Substitution Cipher \*\***

**AIM :** Write a program to perform encryption and decryption using the following algorithms: Ceaser Cipher, Substitution Cipher.

#### **OBJECTIVE:**

1. To understand the concept of Monoalphabetic Cipher Substitution Cipher.
2. Write programs for encryption and decryption using Monoalphabetic Cipher Substitution Cipher.

#### **THEORY:**

Better than Caesar Cipher. If, instead the “cipher” line can be any permutation of the key 26 alphabetic characters, then there are  $26!$  Or greater than  $4 * 10^{26}$  possible keys. This is 10 orders of magnitude greater than the key space for DES and would seem to as a Mono-alphabetic substitution cipher, because a single cipher alphabet is used per message. There is however, another line of attack. If one analytically knows the nature of the plain text, then the analyst can exploit the regularities of the language.

#### **Example**

For each character of alphabet, assign different-abrupt concerned character.

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | c | d | e | f | g | h | i | j | k | l | m |
|---|---|---|---|---|---|---|---|---|---|---|---|---|

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Z | Y | X | W | V | U | T | S | R | Q | P | O | N |
|---|---|---|---|---|---|---|---|---|---|---|---|---|

|   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| n | o | p | q | r | s | t | u | v | w | x | y | z |
| M | L | K | J | I | H | G | F | E | D | C | B | A |

Plain text: goodmorning

**Cipher text: TLLWNLIMRMT**

### Limitations

Monoalphabetic ciphers are easy to break because they reflect the frequency data of the original alphabet. A countermeasure is to provide multiple substitutes, known as homophones, for a single letter.

### Code:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#define pf printf
#define sf scanf
char dc[26]="XDGSZANYOBTMJCEVFHKWPLQURI";
char ec[26]="abcdefghijklmnopqrstuvwxyz";
void e(char*);
void d(char*);
void main(){
    char *p;
    clrscr();
    pf("\n\nEnter plain text:");
    sf("%s",p);
    pf("\nAfter Encryption:\n-----\n");
    e(p);
    pf("\n%s",p);
    pf("\nAfter Decryption:\n-----\n");
    d(p);
```

```
    pf("\n%s",p);
    getch();
}

void e(char *p){
    int l=0;
    while(*(p+l) != '\0'){
        *(p+l)=dc[*(p+l)-97];
        l++;
    }
}

void d(char *p){
    int l=0;
    while(*(p+l) != '\0'){
        int i;
        for(i=0;i<26;i++){
            if(dc[i]==*(p+l))
                break;
        }
        *(p+l)=ec[i];
        l++;
    }
}
```

### Output:



```
File Edit Search Run Compile Debug Project Options Window Help
Output
Enter plain text:
helloworld

After Encryption:
-----
YZMEQEHMS

After Decryption:
-----
helloworld
```

### Conclusion:

---



T.E. (E & TC)

Network Security

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]      SUBJECT: - EL-II-NS      DATE:      /      / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 4

#### **\*\* Implementation of Symmetric and Asymmetric cryptography \*\***

**AIM :** Implementation of Symmetric and Asymmetric cryptography.

#### **OBJECTIVE:**

1. To understand the concept of Asymmetric cryptography.
2. Write programs for encryption and decryption using RSA algorithm.

#### **THEORY:**

##### **Symmetric cryptography and Asymmetric cryptography:**

Symmetric cryptography uses a single key that needs to be shared among the people who need to receive the message while asymmetric cryptography uses a pair of public key and a private key to encrypt and decrypt messages when communicating.

RSA algorithm is based on difficulty of factorizing large number that have 2 and only 2 factors. Works on public and private key system. The public key is made available to everyone, with this key user can encrypt data but cannot decrypt it. The only person who can decrypt it is one who passes the private key.

It is theoretically possible but extremely difficult to generate the private key for the public key. This makes the RSA algorithm very popular choice in data encryption.



**RSA algorithm:****RSA Key Setup:**

- Each user generates a public/private key pair by:
- Selecting two large primes at random -  $p, q$
- Computing their system modulus  $n=p.q$
- note  $m=(p-1)(q-1)$
- selecting at random the encryption key  $e$
- where  $1 < e < m$ ,  $\gcd(e, m) = 1$
- Find  $d$  such that  $1 = de \bmod m$
- $1 = -km + de$
- publish their public encryption key:  $PU = \{e, n\}$
- keep secret private decryption key:  $PR = \{d, n\}$

**Encryption:**

- **To encrypt a message  $M$  the sender:**
- obtains public key of recipient  $PU = \{e, n\}$
- computes:  $C = M^e \bmod n$ , where  $0 \leq M < n$

**Decryption:**

- To decrypt the cipher text  $C$  the recipient:
- uses their private key  $PR = \{d, n\}$
- computes:  $M = C^d \bmod n$

*Note that the message  $M$  must be smaller than the modulus  $n$*

**EXAMPLE:**

We start by selecting primes  $p=3, q=11$

$$n=p*q=33, m=(p-1)(q-1)=(2)(10)=20$$

try  $e=3$

$$\gcd(3,20)=1$$

Plain Text  $P = 9$

$e$  is co-prime form.

Find  $d$  such that  $1=de \bmod m$

$$1=-km + de, \quad \text{Where } k = 1$$

Using the extended eluded algorithm we say,

$$1 = -1(20) + d(3) \quad d=7$$

Now late say that we want to encrypt the no  $x=a$ , we use the encryption function,

$$C = P^e \bmod n$$

$$C = 9^3 \bmod 33$$

$$C = 729 \bmod 33$$

$$C = 3$$

To decrypt we use the function,

$$P = C^d \bmod n$$

$$P = 3^7 \bmod 33$$

$$P = 2187 \bmod 33$$

$$P = 9$$

### **RSA algorithm Code (C Language):**

```
// Program of RSA Alogorithm
```

```
// TE (EnTC) Network Security
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<math.h>
```

```
#include<string.h>
long int p,q,n,t,flag,e[100],d[100],temp[100],j,m[100],en[100],i;
char msg[100];
int prime(long int);
void ce();
long int cd(long int);
void encrypt();
void decrypt();
int main()
{
    printf("\nENTER FIRST PRIME NUMBER\n");
    scanf("%ld",&p);
    flag=prime(p);
    if(flag==0)
    {
        printf("\nWRONG INPUT\n");
        exit(1);
    }
    printf("\nENTER ANOTHER PRIME NUMBER\n");
    scanf("%ld",&q);
    flag=prime(q);
    if(flag==0||p==q)
    {
        printf("\nWRONG INPUT\n");
        exit(1);
    }
    printf("\nENTER MESSAGE\n");
    fflush(stdin);
    scanf("%s",msg);
    for(i=0;msg[i]!=NULL;i++)
        m[i]=msg[i];
    n=p*q;
```

```

t=(p-1)*(q-1);
ce();
printf("\nPOSSIBLE VALUES OF e AND d ARE\n");
for(i=0;i<j-1;i++)
    printf("\n%ld\t%ld",e[i],d[i]);
encrypt();
decrypt();
return 0;
}

```

```

int prime(long int pr)
{
    int i;
    j=sqrt(pr);
    for(i=2;i<=j;i++)
    {
        if(pr%i==0)
            return 0;
    }
    return 1;
}

```

```

void ce()
{
    int k;
    k=0;
    for(i=2;i<t;i++)
    {
        if(t%i==0)
            continue;
        flag=prime(i);
        if(flag==1&&i!=p&&i!=q)
        {
            e[k]=i; flag=cd(e[k]);
            if(flag>0)
            {
                d[k]=flag;
                k++;
            }
        }
        if(k==99)
            break;
    }
}

```

```

}

long int cd(long int x)
{
    long int k=1;
    while(1)
    {
        k=k+t;
        if(k%x==0)
            return(k/x);
    }
}

void encrypt()
{
    long int pt,ct,key=e[0],k,len;
    i=0;
    len=strlen(msg);
    while(i!=len)
    {
        pt=m[i];
        pt=pt-96;
        k=1;
        for(j=0;j<key;j++)
        {
            k=k*pt;
            k=k%n;
        }
        temp[i]=k;
        ct=k+96;
        en[i]=ct;
        i++;
    }
    en[i]=-1;
    printf("\nTHE ENCRYPTED MESSAGE IS\n");
    for(i=0;en[i]!=-1;i++)
        printf("%c",en[i]);
}

void decrypt()
{
    long int pt,ct,key=d[0],k;
    i=0;
    while(en[i]!=-1)
    {
        ct=temp[i];
        k=1;

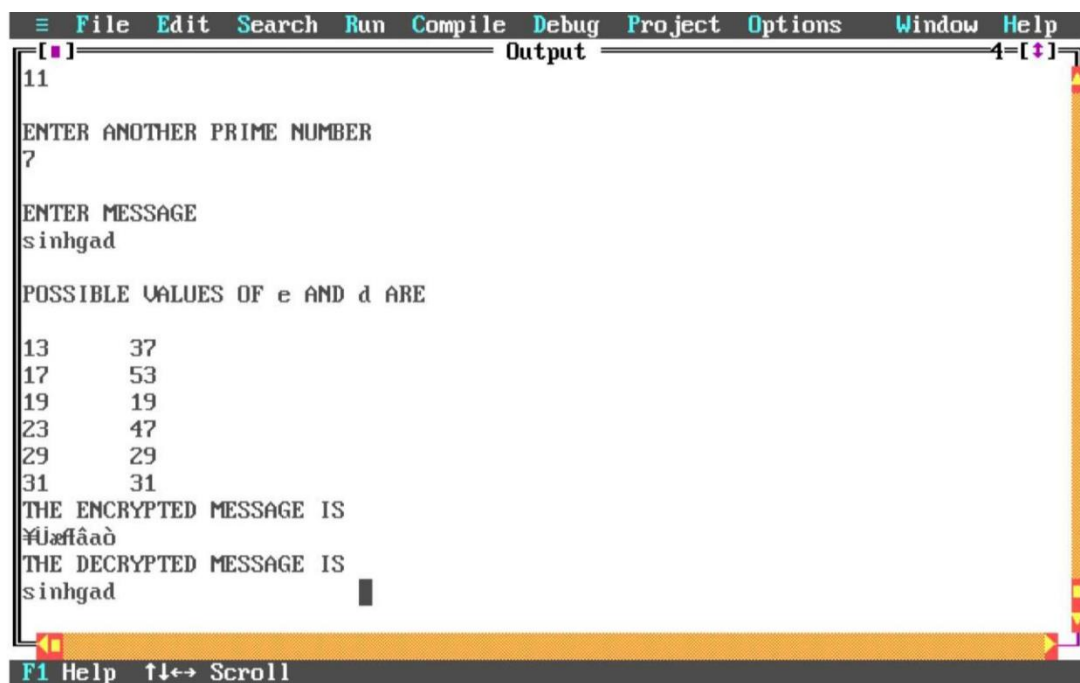
        for(j=0;j<key;j++)
        {

```

```
k=k*ct;
k=k%n;
}
pt=k+96;
m[i]=pt;
i++;
}
m[i]=-1;
printf("\nTHE DECRYPTED MESSAGE IS\n");
for(i=0;m[i]!=-1;i++)
    printf("%c",m[i]);
}
```

## Output:

### RSA algorithm Output:



```
11
ENTER ANOTHER PRIME NUMBER
7
ENTER MESSAGE
sinhgad
POSSIBLE VALUES OF e AND d ARE
13      37
17      53
19      19
23      47
29      29
31      31
THE ENCRYPTED MESSAGE IS
ÿÿÿÿÿÿÿÿ
THE DECRYPTED MESSAGE IS
sinhgad
```

## Conclusion:

---

T.E. (E & TC)

Network Security

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]      SUBJECT: - EL-II-NS      DATE:      /      / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 5

#### **\*\* Implementation of Symmetric and Asymmetric cryptography \*\***

**AIM :** Implementation of Symmetric cryptography.

#### **OBJECTIVE:**

1. To understand the concept of Symmetric cryptography.
2. Write programs for Diffie-Hellman key agreement.

#### **THEORY:**

##### **Symmetric cryptography and Asymmetric cryptography:**

Symmetric cryptography uses a single key that needs to be shared among the people who need to receive the message while asymmetric cryptography uses a pair of public key and a private key to encrypt and decrypt messages when communicating.

##### **Diffie-Hellman key agreement:**

Diffie-Hellman protocol allows two communication parties; say Alice and Bob to create a Symmetric session key without need of KDC (key distribution centre)

##### **Diffie-Hellman protocol:**

Alice and Bob choose two numbers P&Q which are public.



'p' is large prime of the order of 1024 bits.

'g' is a generator of order R1 in the group  $2^*p$

Alice chooses a large random number 'x' in the range 0 to p-1 and calculates  $R1 = g^x \text{ mod } p$ .

Bob chooses a large random number 'y' in the range 0 to p-1 and calculates  $R2 = g^y \text{ mod } p$ .

Alice sends R1 to Bob and Bob sends R2 to Alice

Alice calculates  $KA = (R2)^x \text{ mod } p$

Bob calculates  $KB = (R1)^y \text{ mod } p$

$K = (9n \text{ mod } p)y \text{ mod } p = (g^y \text{ mod } p)^x \text{ mod } p$

$K = g^{xy} \text{ mod } p$ .

Where K is the symmetric key for the session.

**Example:**

- $g=7$
- $p=23$
- $x=3$
- $y=6$
- 

**Calculate:**

1.  $R1 = g^x \text{ mod } p$ .

2.  $R2 = g^y \text{ mod } p$ .

3. Alice Key  $k_A = (R2)^x \text{ mod } p$

4. Bob Key  $k_B = (R1)^y \text{ mod } p$

5.  $K = g^{xy} \text{ mod } p$ .

**Answers:**

1.  $R1 = g^x \bmod p = 7^3 \bmod 23 = 343 \bmod 23 = 21$

2.  $R2 = g^y \bmod p = 7^6 \bmod 23 = 1,17,649 \bmod 23 = 4$

3. Alice Key  $k_A = (R2)^x \bmod p = 4^3 \bmod 23 = 18$

4. Bob Key  $k_B = (R1)^y \bmod p = 21^6 \bmod 23 = 18$

5.  $K = g^{xy} \bmod p = 7^{3 \times 6} \bmod p = 18$

**1. Diffie-Hellman key agreement Code (Scilab Language):**

```
clc;
```

```
g=7;
```

```
p=23;
```

```
printf("\n The results are as follows:\n\n");
```

```
x=3;
```

```
y=6;
```

```
R1=modulo(g^x,p);
```

```
R2=modulo(g^y,p);
```

```
printf("1)Alice choose x=%d & calculates R1=%d\n\n2)Bob chooses y=%d & calculates R2=%d\n\n3)Alice sends the number %d to Bob \n\n4)Bob sends the number %d to Alice \n\n",x,R1,y,R2,R1,R2);
```

```
K_Alice =modulo((R2)^x,p);
```

```
K_Bob=modulo((R1)^y,p);
```

```
K_Final=modulo(g^(x*y),p);
```

```
printf('5)Alice calculates the symmetric key K=%d \n\n6)Bob calculates the symmetric key  
k=%d\n\n7)K_Final=%d \n\n',K_Alice,K_Bob,K_Final);
```

## Output:

### Diffie-Hellman key agreement Output:

```
Result:
The results are as follows:
1)Alice choose x=3 & calculates R1=21
2)Bob chooses y=6 & calculates R2=4
3)Alice sends the number 21 to Bob
4)Bob sends the number 4 to Alice
5)Alice calculates the symmetric key K=18
6)Bob calculates the symmetric key k=18
7)K_Final=18
```

## Conclusion:

---

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]    SUBJECT: - EL-II-NS    DATE:        /        / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 6

\*\* Implementation of DES \*\*

**AIM : Implementation of DES.**

#### **OBJECTIVE:**

1. To understand the concept of block cipher and symmetric key cryptography.
2. Write program for encryption and decryption using DES algorithm

#### **THEORY:**

##### **Description of DES:**

DES is a block cipher; it encrypts data in 64-bit blocks. A 64-bit block of plaintext goes in one end of the algorithm and a 64-bit block of cipher text comes out the other end. DES is a symmetric algorithm: The same algorithm and key are used for both encryption and decryption, the key length is 56 bits. (The key is usually expressed as a 64-bit number, but every eighth bit is used for parity checking and is ignored. These parity bits are the least-significant bits of the key bytes. The key can be any 56-bit number and can be changed at any time.

##### **Outline of the Algorithm:**

DES operates on a 64-bit block of plaintext. After an initial permutation, the block is broken into a right half and a left half, each 32 bits long. Then there are 16 rounds of identical operations called Function  $f$ , in which the data are combined with the key. After the sixteenth round, the right and left halves are joined, and a final permutation (the inverse of the initial permutation) finishes off the algorithm. In each round (see Figure 1'), the key bits are shifted, and then 48 bits are selected from the 56 bits of the key. The right half of the data is expanded to 48 bits via an expansion permutation, combined with 48 bits of a shifted and permuted key via an XOR, sent through 8 S-boxes producing 32 new bits, and permuted again. These four operations make up Function  $f$ . The output of Function  $f$  is then combined with the left half via another XOR. The result of these operations becomes the new half; the old half becomes the new left half. These operations are repeated 16 times, making 16 rounds of DES.

If  $B_i$  is the result of the  $i$ th iteration,  $L_i$  and  $R_i$  are the left and right halves of  $B_i$ ,  $K_i$  is the 48-bit key for round  $i$ , and  $f$  is the function that does all the substituting and permuting and XORing via the key, then a round looks like:

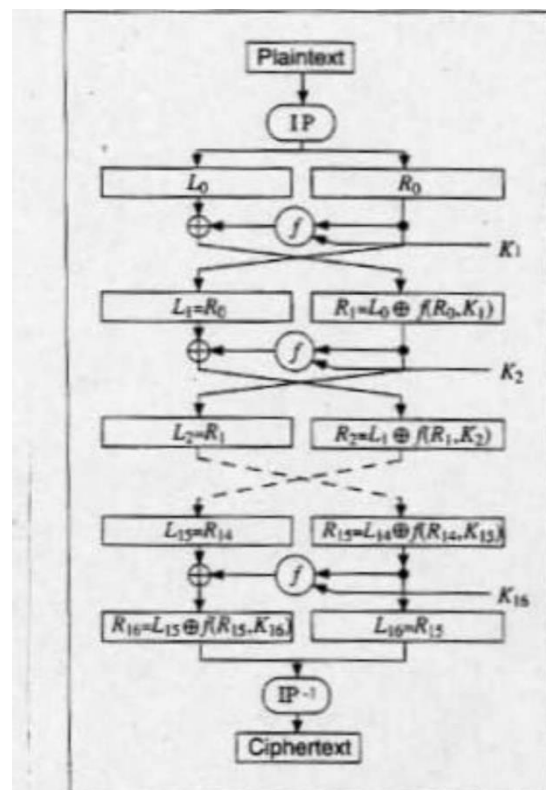
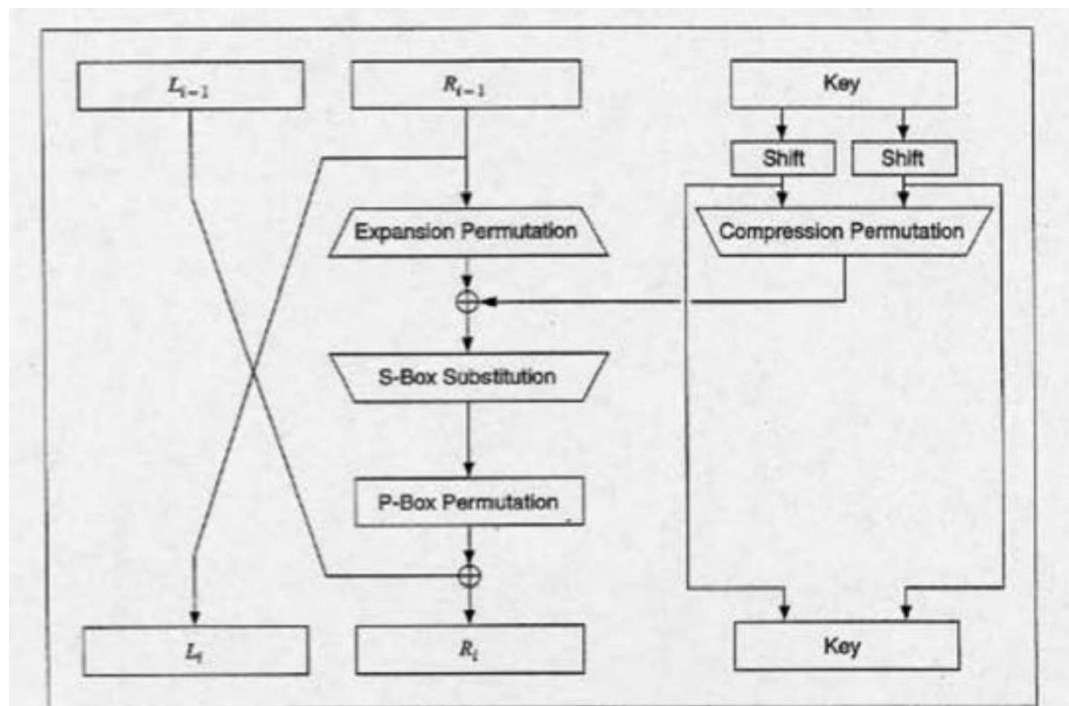


Fig 1.DES algorithm

### The Initial Permutation:

The initial permutation occurs before round 1; For example the initial permutation moves bit 58 of the plaintext to bit position 1, bit 50 to bit position 2, bit 42 to bit position 3, and so on. The initial permutation and the corresponding final permutation do not affect DES's security. Its primary purpose is to make it easier to load plaintext and cipher text data into a DES chip in byte-sized pieces.



### The Expansion Permutation:

This operation expands the half of the data,  $R_i$ . From 32 bits to 48 bits. Because this operation changes the order of the bits as well as repeating certain bits, it is known as an expansion permutation.

### The S-BOX Substitution:

After the compressed key is XORed with the expanded block, the 48-bit result moves to a substitution operation. The substitutions are performed by eight substitution boxes, or S-boxes. Each S-box has a 6-bit input and a 4-bit output, and there are different S-boxes. (The total memory requirement for the eight DES S-boxes is 256 bytes.) The 48 bits are divided into eight

6-bit sub-blocks. Each separate block is operated on by a separate S-box: The first block is operated on by S-box 1, the second block is operated on by S-box 2, and so on.

Each S-box is a table of 4 rows and 16 columns. Each entry in the box is 16 bit number. The 6 input bits of the S-box specify under which row and column number to look for the output.

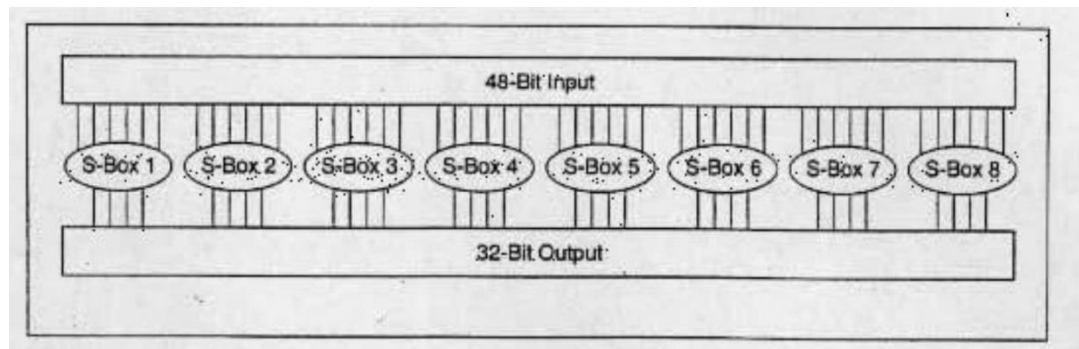


Fig 2. S-boxes

### The P-Box Permutation

The 32-bit output of the S-box substitutions is permuted according to a P-box. This permutation maps each input bit to an output position; no bits are used twice and no bits are ignored. This is called a straight permutation or just a permutation, the position to which each bit moves. For example, bit 21 moves to bit 4, while bit 4 moves to bit 31.

### The Final Permutation

The final permutation is the inverse of the initial permutation. The left and right halves are not exchanged after the last round of DES; instead, the concatenated block R16L16 is used as the input to the final permutation. There's nothing going on exchanging the halves and shifting around the permutation would yield exactly the same result. This is so that the algorithm can be used to both encrypt and decrypt.

### DES Algorithm Code (Python Programming Language):

#### # DES Algorithm

# Enter the Plain Text & Key -----

```
print("\n-----\n")
print("DES - DATA ENCRYPTION STANDARD ALGORITHM ")
pt = input("\nEnter the 64 byte Plain Text (16 Characters) : ")
key = input("\nEnter the 64 byte key (16 Characters) : ")

#-----

#----- Start of DES Algorithm #

# Hexadecimal to binary conversion
def hex2bin(s):
    mp = {'0': "0000",
          '1': "0001",
          '2': "0010",
          '3': "0011",
          '4': "0100",
          '5': "0101",
          '6': "0110",
          '7': "0111",
          '8': "1000",
          '9': "1001",
          'A': "1010",
          'B': "1011",
          'C': "1100",
          'D': "1101",
          'E': "1110",
          'F': "1111" }
    bin = ""
    for i in range(len(s)):
        bin = bin + mp[s[i]]
    return bin

# Binary to hexadecimal conversion
def bin2hex(s):
    mp = {"0000": '0',
          "0001": '1',
          "0010": '2',
          "0011": '3',
          "0100": '4',
          "0101": '5',
          "0110": '6',
          "0111": '7',
          "1000": '8',
          "1001": '9',
          "1010": 'A',
          "1011": 'B',
          "1100": 'C',
          "1101": 'D',
          "1110": 'E',
          "1111": 'F' }
    hex = ""
```



```
for i in range(0,len(s),4):
    ch = ""
    ch = ch + s[i]
    ch = ch + s[i + 1]
    ch = ch + s[i + 2]
    ch = ch + s[i + 3]
    hex = hex + mp[ch]

return hex

# Binary to decimal conversion
def bin2dec(binary):

    binary1 = binary
    decimal, i, n = 0, 0, 0
    while(binary != 0):
        dec = binary % 10
        decimal = decimal + dec * pow(2, i)
        binary = binary//10
        i += 1
    return decimal

# Decimal to binary conversion
def dec2bin(num):
    res = bin(num).replace("0b", "")
    if(len(res)%4 != 0):
        div = len(res) / 4
        div = int(div)
        counter =(4 * (div + 1)) - len(res)
        for i in range(0, counter):
            res = '0' + res
    return res

# Permute function to rearrange the bits
def permute(k, arr, n):
    permutation = ""
    for i in range(0, n):
        permutation = permutation + k[arr[i] - 1]
    return permutation

# shifting the bits towards left by nth shifts
def shift_left(k, nth_shifts):
    s = ""
    for i in range(nth_shifts):
        for j in range(1,len(k)):
            s = s + k[j]
        s = s + k[0]
        k = s
        s = ""
    return k

# calculating xow of two strings of binary number a and b
```

```
def xor(a, b):
    ans = ""
    for i in range(len(a)):
        if a[i] == b[i]:
            ans = ans + "0"
        else:
            ans = ans + "1"
    return ans

# Table of Position of 64 bits at initial level: Initial Permutation Table
initial_perm = [58, 50, 42, 34, 26, 18, 10, 2,
                60, 52, 44, 36, 28, 20, 12, 4,
                62, 54, 46, 38, 30, 22, 14, 6,
                64, 56, 48, 40, 32, 24, 16, 8,
                57, 49, 41, 33, 25, 17, 9, 1,
                59, 51, 43, 35, 27, 19, 11, 3,
                61, 53, 45, 37, 29, 21, 13, 5,
                63, 55, 47, 39, 31, 23, 15, 7]

# Expansion D-box Table
exp_d = [32, 1, 2, 3, 4, 5, 4, 5,
         6, 7, 8, 9, 8, 9, 10, 11,
         12, 13, 12, 13, 14, 15, 16, 17,
         16, 17, 18, 19, 20, 21, 20, 21,
         22, 23, 24, 25, 24, 25, 26, 27,
         28, 29, 28, 29, 30, 31, 32, 1 ]

# Straight Permutation Table
per = [ 16, 7, 20, 21,
        29, 12, 28, 17,
        1, 15, 23, 26,
        5, 18, 31, 10,
        2, 8, 24, 14,
        32, 27, 3, 9,
        19, 13, 30, 6,
        22, 11, 4, 25 ]

# S-box Table
sbox = [[[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
         [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
         [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
         [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13 ]],

        [[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
         [3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
         [0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
         [13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9 ]],

        [[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
         [13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
         [13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
         [1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12 ]]]
```

```
[ [7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
[13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
[10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
  [3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14] ],

[ [2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
[14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],
  [4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
[11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3] ],

[ [12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
[10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
  [9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
  [4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13] ],

[ [4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
[13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
  [1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
  [6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12] ],

[ [13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
  [1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
  [7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
  [2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11] ] ]
```

# Final Permutation Table

```
final_perm = [ 40, 8, 48, 16, 56, 24, 64, 32,
               39, 7, 47, 15, 55, 23, 63, 31,
               38, 6, 46, 14, 54, 22, 62, 30,
               37, 5, 45, 13, 53, 21, 61, 29,
               36, 4, 44, 12, 52, 20, 60, 28,
               35, 3, 43, 11, 51, 19, 59, 27,
               34, 2, 42, 10, 50, 18, 58, 26,
               33, 1, 41, 9, 49, 17, 57, 25 ]
```

def encrypt(pt, rkb, rk):

pt = hex2bin(pt)

# Initial Permutation

pt = permute(pt, initial\_perm, 64)

print("\nAfter initial permutation", bin2hex(pt))

print("\n")

# Splitting

left = pt[0:32]

right = pt[32:64]

for i in range(0, 16):

# Expansion D-box: Expanding the 32 bits data into 48 bits

right\_expanded = permute(right, exp\_d, 48)

# XOR RoundKey[i] and right\_expanded

xor\_x = xor(right\_expanded, rkb[i])

```
# S-boxex: substituting the value from s-box table by calculating row and column
sbox_str = ""
for j in range(0, 8):
    row = bin2dec(int(xor_x[j * 6] + xor_x[j * 6 + 5]))
    col = bin2dec(int(xor_x[j * 6 + 1] + xor_x[j * 6 + 2] + xor_x[j * 6 + 3] + xor_x[j * 6
+ 4]))
    val = sbox[j][row][col]
    sbox_str = sbox_str + dec2bin(val)

# Straight D-box: After substituting rearranging the bits
sbox_str = permute(sbox_str, per, 32)

# XOR left and sbox_str
result = xor(left, sbox_str)
left = result

# Swapper
if(i != 15):
    left, right = right, left
print("Round ", i + 1, " ", bin2hex(left), " ", bin2hex(right), " ", rk[i])

# Combination
combine = left + right

# Final permutation: final rearranging of bits to get cipher text
cipher_text = permute(combine, final_perm, 64)
return cipher_text

# Key generation
# --hex to binary
key = hex2bin(key)

# --parity bit drop table
keyp = [57, 49, 41, 33, 25, 17, 9,
        1, 58, 50, 42, 34, 26, 18,
        10, 2, 59, 51, 43, 35, 27,
        19, 11, 3, 60, 52, 44, 36,
        63, 55, 47, 39, 31, 23, 15,
        7, 62, 54, 46, 38, 30, 22,
        14, 6, 61, 53, 45, 37, 29,
        21, 13, 5, 28, 20, 12, 4 ]

# getting 56 bit key from 64 bit using the parity bits
key = permute(key, keyp, 56)

# Number of bit shifts
shift_table = [1, 1, 2, 2,
               2, 2, 2, 2,
               1, 2, 2, 2,
               2, 2, 2, 1 ]
```

# Key- Compression Table : Compression of key from 56 bits to 48 bits

```
key_comp = [14, 17, 11, 24, 1, 5,
            3, 28, 15, 6, 21, 10,
            23, 19, 12, 4, 26, 8,
            16, 7, 27, 20, 13, 2,
            41, 52, 31, 37, 47, 55,
            30, 40, 51, 45, 33, 48,
            44, 49, 39, 56, 34, 53,
            46, 42, 50, 36, 29, 32 ]
```

# Splitting

left = key[0:28] # rkb for RoundKeys in binary

right = key[28:56] # rk for RoundKeys in hexadecimal

rkb = []

rk = []

for i in range(0, 16):

    # Shifting the bits by nth shifts by checking from shift table

    left = shift\_left(left, shift\_table[i])

    right = shift\_left(right, shift\_table[i])

    # Combination of left and right string

    combine\_str = left + right

    # Compression of key from 56 to 48 bits

    round\_key = permute(combine\_str, key\_comp, 48)

    rkb.append(round\_key)

    rk.append(bin2hex(round\_key))

#-----End of DES -----

print("\nNow performing Encryption")

cipher\_text = bin2hex(encrypt(pt, rkb, rk))

print("\nCipher Text : ", cipher\_text)

print("\n-----\n")

print("\nNow performing Decryption")

rkb\_rev = rkb[::-1]

rk\_rev = rk[::-1]

text = bin2hex(encrypt(cipher\_text, rkb\_rev, rk\_rev))

print("\nPlain Text : ", text)

print("\n----- End of DES Program -----")

#-----

### DES Algorithm Output:

```
Shell Clear
DES - DATA ENCRYPTION STANDARD ALGORITHM
Enter the 64 byte Plain Text (16 Characters) : 0123456789ABCDEF
Enter the 64 byte key (16 Characters) : 0123456789ABCDEF
Now performing Encryption
After initial permutation CC00CCFF0AAFOAA

Round 1  F0AAF0AA  5E1CEC63  0B02679B49A5
Round 2  5E1CEC63  82E13C49  69A659256A26
Round 3  82E13C49  499542F9  45D48AB428D2
Round 4  499542F9  0DD64AFB  7289D2A58257
Round 5  0DD64AFB  7036043B  3CE80317A6C2
Round 6  7036043B  F1470BC2  23251E3C8545
Round 7  F1470BC2  394C8F45  6C04950AE4C6
Round 8  394C8F45  348DC746  5788386CE581
Round 9  348DC746  F37100C6  C0C9E926B839
Round 10 F37100C6  3C22A9CB  91E307631D72
Round 11 3C22A9CB  0A37C369  211F830D893A
Round 12 0A37C369  5C725FFB  7130E5455C54
Round 13 5C725FFB  F4748AD6  91C4D04980FC
Round 14 F4748AD6  CC6C340E  5443B681DC8D
Round 15 CC6C340E  BA88F699  B691050A16B5
Round 16 FB21FB9C  BA88F699  CA3D03887032
```

```
Shell
Cipher Text : 56CC09E7CFDC4CEF
-----
Now performing Decryption
After initial permutation FB21FB9CBA88F699
Round 1  BA88F699  CC6C340E  CA3D03887032
Round 2  CC6C340E  F4748AD6  B691050A16B5
Round 3  F4748AD6  5C725FFB  5443B6810C8D
Round 4  5C725FFB  0A37C369  91C4D04980FC
Round 5  0A37C369  3C22A9CB  7130E5455C54
Round 6  3C22A9CB  F37100C6  211F8300893A
Round 7  F37100C6  348DC746  91E307631D72
Round 8  348DC746  394C8F45  C0C9E9268839
Round 9  394C8F45  F1470BC2  5788386CE581
Round 10 F1470BC2  7036043B  6C04950AE4C6
Round 11 7036043B  00D64AFB  23251E3C8545
Round 12 00D64AFB  499542F9  3CE80317A6C2
Round 13 499542F9  82E13C49  728902A58257
Round 14 82E13C49  5E1CEC63  45D48AB428D2
Round 15 5E1CEC63  F0AAF0AA  69A659256A26
Round 16 CC00CCFF  F0AAF0AA  0B02679849A5
Plain Text : 0123456789ABCDEF
```

## Conclusion:

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]      SUBJECT: - EL-II-NS      DATE:      /      / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 7 \*\* Steganography \*\*

**AIM :** Implementation of Steganography.

**OBJECTIVE:**

1. To understand the concept of Steganography.
2. Write programs for concealing a message within another message (image file).

**TOOLS:** Xiao Stenography tool

**THEORY:**

Steganography is the process of hiding a secret message within a larger one in such a way that someone cannot know the presence or contents of the hidden message. The purpose of Steganography is to maintain secret communication between two parties. Unlike cryptography, which conceals the contents of a secret message, steganography conceals the very fact that a message is communicated. Although steganography differs from cryptography, there are many analogies between the two, and some authors classify steganography as a form of cryptography since hidden communication is a type of secret message.

**Advantage of using Steganography over Cryptography?**

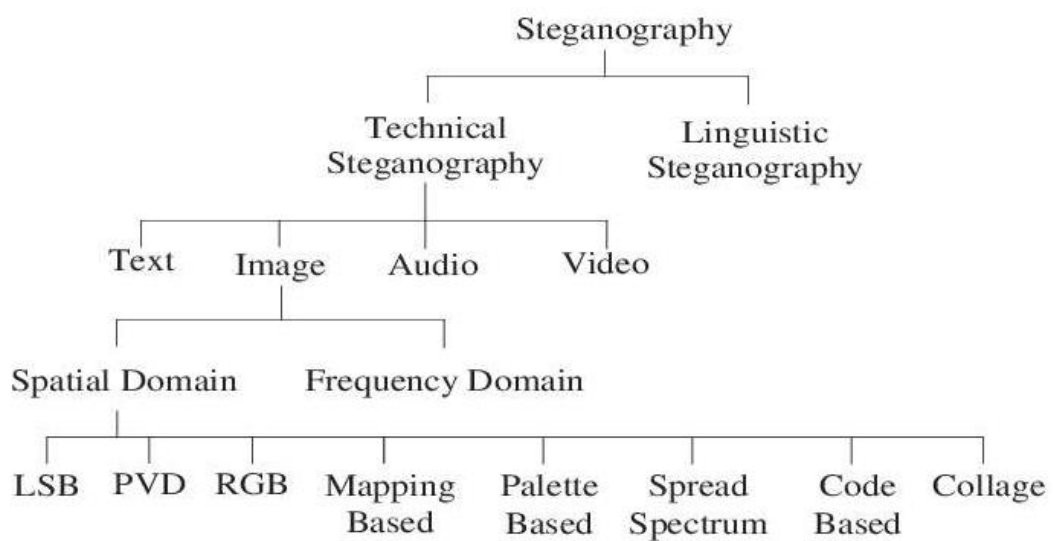
Up to now, cryptography has always had its ultimate role in protecting the secrecy between the sender and the intended receiver. However, nowadays steganography techniques are used increasingly besides cryptography to add more protective layers to the hidden data. The advantage of using steganography over cryptography alone is that the intended secret message does not attract



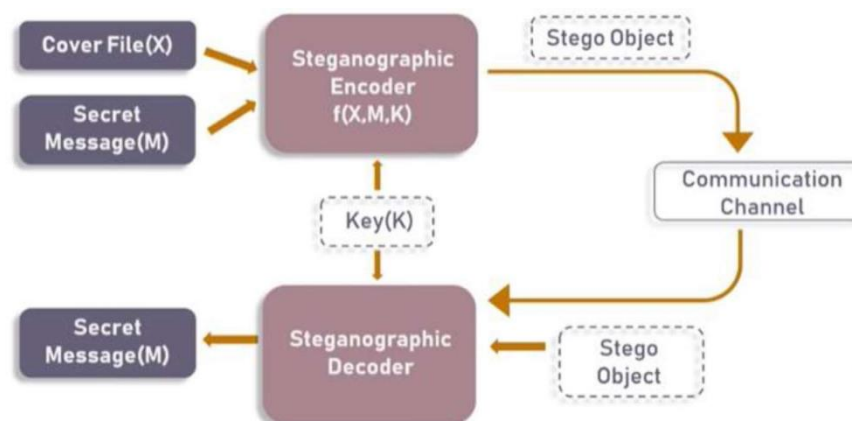
attention to itself as an object of scrutiny. Plainly visible encrypted messages, no matter how unbreakable they are, arouse interest and may in themselves be incriminating in countries in which encryption is illegal.

## Types of Steganography

Steganography works have been carried out on different transmission media like images, video, text, or audio.



## Basic Steganographic Model



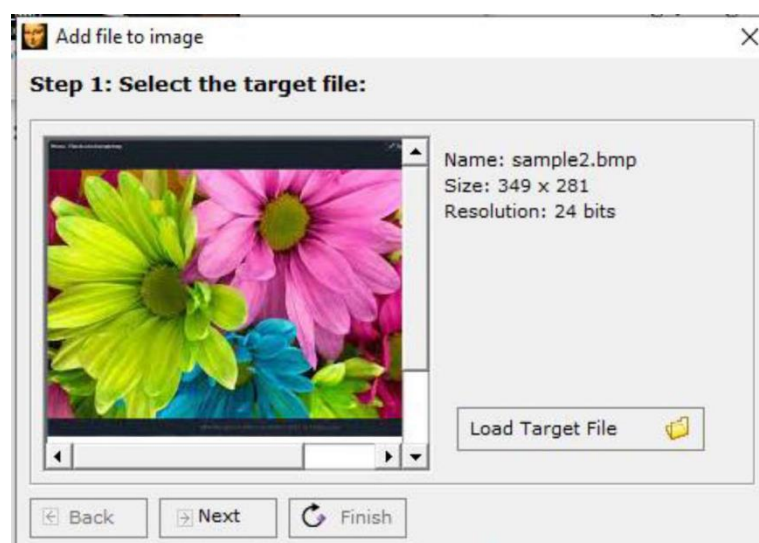
As seen in the above image, both the original image file(X) and secret message (M) that needs to be hidden are fed into a steganographic encoder as input. Steganographic Encoder function,  $f(X,M,K)$  embeds the secret message into a cover image file by using techniques like least significant bit encoding. The resulting stego image looks very similar to your cover image file, with no visible changes. This completes encoding. To retrieve the secret message, stego object is fed into Steganographic Decoder.

## Procedure:

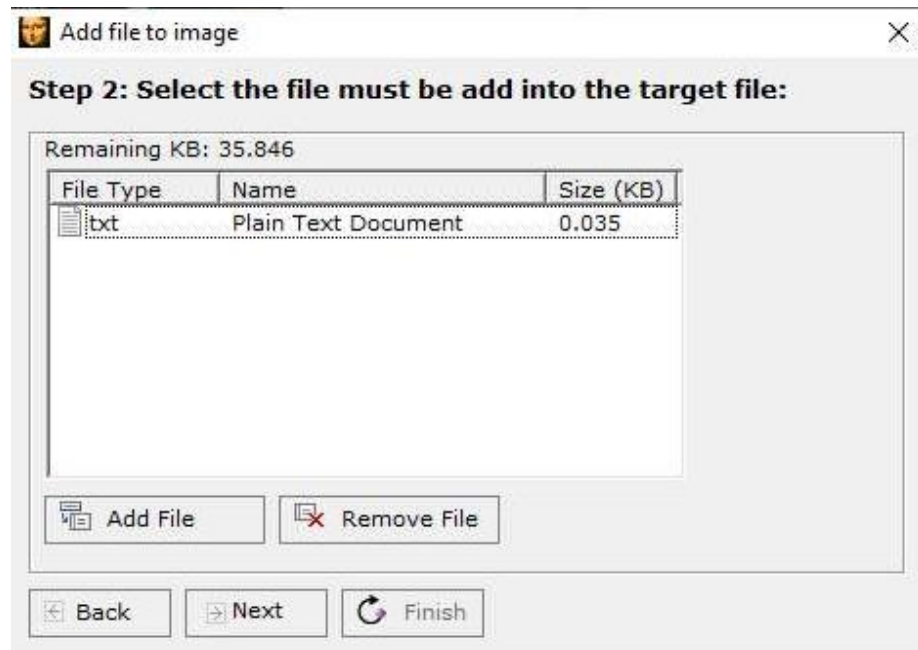
- 1) Start the latest version of any steganography tool.



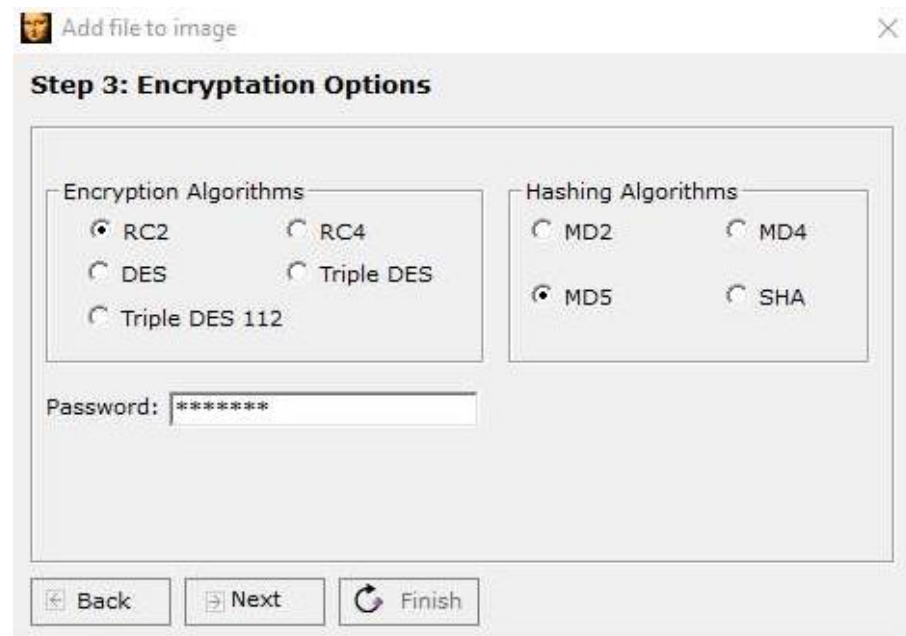
- 2) Load any target image, which is used to hide the plain text (Secret message). Press Next.



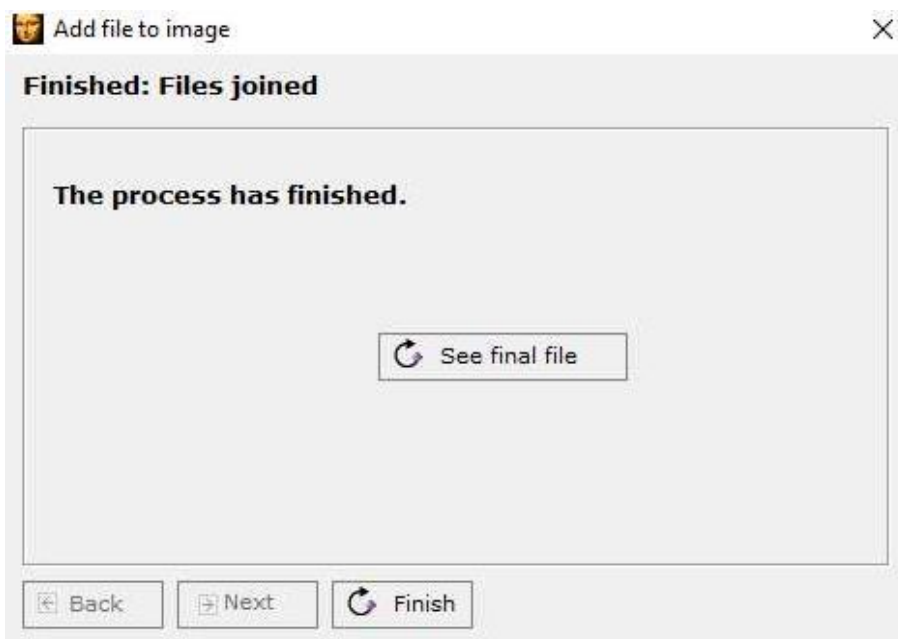
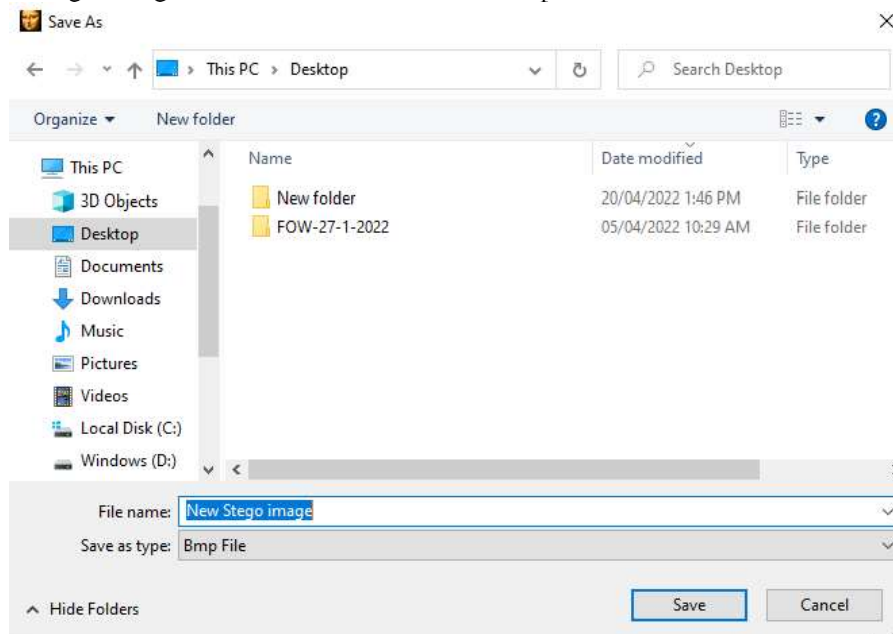
3) Add plain text file to the image file. Press Next



4) Choose one of the given encryption and hashing algorithm. Also give password for the stego image. Press next.

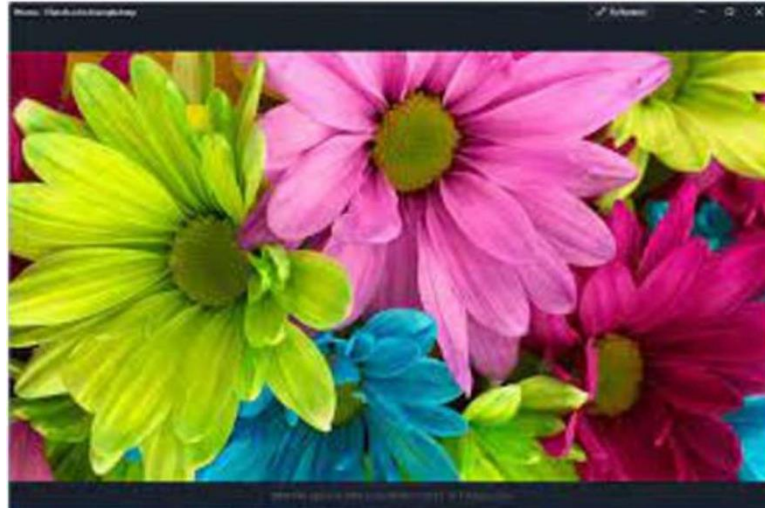


5) Save the new stego image file at desired location. The process is finished.

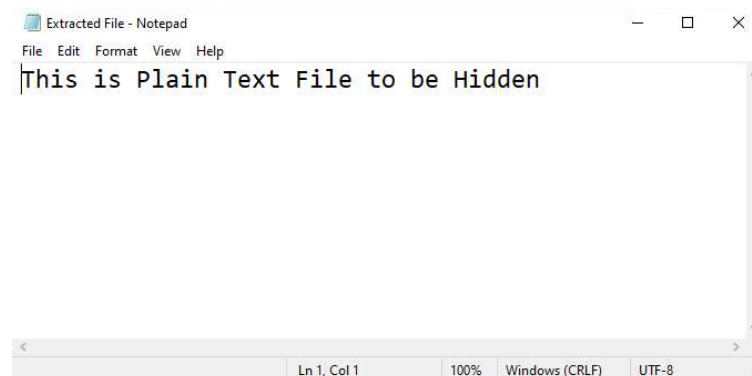
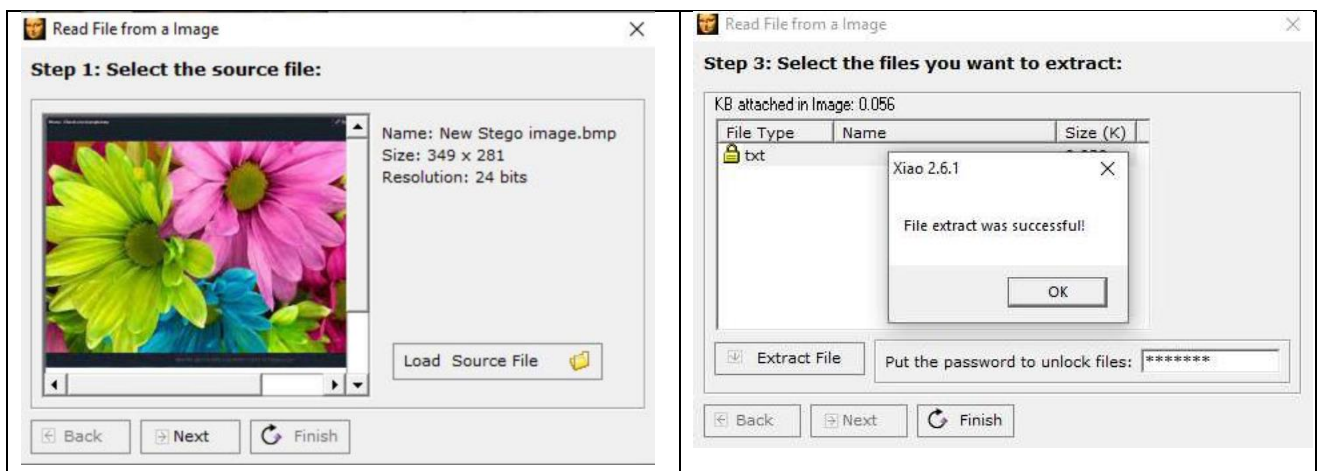


**Output:**

**Encrypted image:**



**Decrypted file:**





T.E. (ETC)

Network Security

**Conclusion:**

---

---

---

---

---

## SEMESTER - II

CLASS: - T.E [ETC]      SUBJECT: - EL-II-NS      DATE:      /      / 20

Roll no: \_\_\_\_\_

Name of the Student: \_\_\_\_\_

### EXPERIMENT NO. 8 \*\* Web Browser Security \*\*

**AIM :** Steps to ensure Security of any one web browser (Mozilla Firefox/Google Chrome).

#### **OBJECTIVE:**

1. To understand the concept of web security.
2. Secure **Mozilla Firefox** web browser for various aspects.

**TOOLS:** Mozilla Firefox, uBlock Origin (open-source, cross-platform browser extension)

#### **THEORY:**

To provide the security to this internet various protocols have been used like SSL (Secure Socket Layer), TLS (Transport Layer Security), etc.

##### **a) SSL Protocol**

SSL Protocol stands for Secure Socket Layer protocol, which is an internet security protocol used for exchanging the information between a web browser and a web server in a secure manner. It provides two basic security services like authentication and confidentiality. SSL protocol has become the world's most popular web security mechanism, all major web browsers support SSL. Secure socket layer protocol is considered as an additional layer in TCP/IP protocol



suite. It is located between the application layer and the transport layer. SSL has three sub protocols namely Handshake Protocol, Record Protocol, and Alert Protocol.

#### **b) TLS Protocol**

TLS stands for Transport Layer Security, which is an internet security protocol. Transport layer security protocol uses a pseudo random function to create a master secret. TLS also has three sub protocols same as SSL protocol – Handshake Protocol, Record Protocol, and Alert Protocol. In Handshake Protocol some details are changed, Record Protocol uses HMAC, Alert protocol newly added features like record overflow, Unknown CA, Decryption failed, Decode error, Access denied, Export restrictions, Protocol version, insufficient security, internal error. Transport layer security is defined in RFC 2246.

#### **c) SHTTP**

SHTTP stands for Secure Hyper Text Transfer Protocol, is a set of security mechanism defined for protecting internet traffic. It also includes data entry forms and internet based transaction. Secure Hyper Text Transfer Protocol works at the application layer, and therefore tightly coupled with HTTP. SHTTP supports both authentication and encryption of HTTP traffic between the client and the server.

#### **d) SET Protocol**

SET Protocol stands for Secure Electronic Transaction protocol is an open encryption and security mechanism designed for protecting the eCommerce transaction over the internet. SET is not a payment system, it is a security protocol used over the internet for secure transaction.

#### **e) PEM Protocol**

PEM Protocol stands for privacy enhanced mail, used for email security over the internet. Enhanced Mail protocol is described in four specific documents RFC 1421, RFC 1422, RFC 1423, and RFC 1424. It supports cryptographic functions namely encryption, nonrepudiation, and message integrity.



## **f) PGP Protocol**

PGP Protocol stands for Pretty Good Privacy, which we developed by Phil Zimmerman. PGP protocol is easy to use and free including its source code documentation. It also supports the basic requirements of cryptography. PGP protocol becomes extremely popular and more widely used as compared to PEM protocol. PGP protocol support cryptography like encryption, Non-repudiation, and message integrity.

### **1. Web Browser Security**

Web browser security consists of all measures, procedures, and policies necessary to protect users accessing the Internet from a web browser application. Almost everyone who is online has a web browser available on their computer or mobile device. Since it is so common, hackers and other cybercriminals prefer to launch compromising attacks on this client-side application. A web browser can store information for your convenience, but others may eventually access the information. Therefore, it provides a large surface area for exposure to email accounts, usernames, all sorts of passwords, and personal or corporate information. Attackers often target the web browser to hijack or sniff on the web traffic from it. They may also use it as a means to access the device itself or any files available on it.

### **2. Why Secure Your Browser**

Today, web browsers such as Microsoft Internet Explorer, Mozilla Firefox, and Apple Safari are installed on almost all computers. Because web browsers are used so frequently, it is vital to configure them securely. Often, the web browser that comes with an operating system is not set up in a secure default configuration. Not securing your web browser can lead quickly to a variety of computer problems caused by anything from spyware being installed without your knowledge to intruders taking control of your computer.

Ideally, computer users should evaluate the risks from the software they use. Many computers are sold with software already loaded. Whether installed by a computer manufacturer, operating system maker, Internet service provider, or by a retail store, the first step in assessing the

vulnerability of your computer is to find out what software is installed and how programs will interact with each other. Unfortunately, it is not practical for most people to perform this level of analysis.

There is an increasing threat from software attacks that take advantage of vulnerable web browsers. We have observed new software vulnerabilities being exploited and directed at web browsers through use of compromised or malicious websites. This problem is made worse by a number of factors, including the following:

- Many users have a tendency to click on links without considering the risks of their actions.
- Web page addresses can be disguised or take you to an unexpected site.
- Many web browsers are configured to provide increased functionality at the cost of decreased security.
- New security vulnerabilities are often discovered after the software is configured.
- Computer systems and software packages may be bundled with additional software, which increases the number of vulnerabilities that may be attacked.
- Third-party software may not have a mechanism for receiving security updates.
- Many websites require that users enable certain features or install more software, putting the computer at additional risk.
- Many users do not know how to configure their web browsers securely.
- Many users are unwilling to enable or disable functionality as required to secure their web browser.

As a result, exploiting vulnerabilities in web browsers has become a popular way for attackers to compromise computer systems.

### **3. How Attackers Target the Browser**

The web browser can display text documents, play multimedia files, and allow users to play games or interact with forms and all other content on the Internet. The versatility of the web browser is good, but this also makes it more challenging to secure since there are more “weak points” an attacker could exploit. The most vulnerable parts of a web browser are as follows:

- a) Connections to DNS servers, websites, and other online resources

A DNS server is the bridge between the browser and the content from any site. It points the browser to the correct website, and the site makes the appropriate content available to the browser. Many attacks compromise and intercept this communication, and it can occur at one of several points. The goal is often to redirect the browser to a malicious website, where the browser (and by implication, the user) encounters drive by downloads, exploit kits, and unwanted content.

#### b) Browser plugins

Browsers are frameworks on which users can install third-party tools to be more comprehensive. However, such plugins may contain vulnerabilities that cyber-attacks can exploit to snoop on the browser's web traffic, hijack it, and install malware or carry out harmful actions on the device. Finance-related data is lucrative for such browser attacks.

#### c) Browser-specific vulnerabilities

Flaws in a browser can enable attackers to sniff sensitive data passing through the web browser, such as when the user fills web forms. These flaws may also give criminal elements unwarranted access to devices.

### 4. How to Improve Web Browser Security

In computing circles, this is also called “hardening the browser” by taking measures to improve security and prevent attacks. Note that it's nearly impossible to achieve 100% impenetrability, but attackers will have a much harder time succeeding.

#### a) Use the latest web browser version

Users should get the latest updates of their web browser software. Vendors often release updates of their browsers, adding new functionality or improving existing features. The most critical security features include:

- **Anti-phishing:** Assess and filter suspicious links in search results or on a webpage.
- **Anti-malware:** Scan and block downloading of suspicious files.
- **Plugin security:** Analyze and block insecure plugins.

- **Sandbox:** Build a fence around web browser processes to prevent access to the operating system.

A few browsers include an auto-update function, notifying the user of updates.

b) Restrict user access

It is advisable to use the web browser from a limited user account without administrator privileges. It limits the ability of any malware that succeeds in infecting the machine to have little to no room to operate within the machine.

c) Use custom security settings

Even though the controls differ, modern browsers often allow some customisation of security-related settings. The recommendation is to set the following settings as high as possible:

- **Block fake sites:** Always enable this feature to prevent unplanned visits to malicious websites.
- **Camera & Microphone:** These should never run automatically. The browser should confirm if the user wants to use the camera or microphone at any time.
- **Cookies:** Completely disable cookies. Users should only enable cookies if a trusted site needs them.
- **JavaScript:** Same rule as for cookies.
- **Plugins/Add-ons:** Same rule as for cookies and JavaScript
- **Pop-up windows:** Same rule as for cookies, JavaScript, and Plugins/Add-ons.

d) Only include plugins that improve security or those you will use

The best thing users can do with plugins is to remove them if they are not using them regularly. With a few clicks, it is easy to re-enable or re-install them when needed. The advantage of removing them is that they can minimize potential points for compromise. Some plugins improve web browser security. Security professionals recommend the following for all browsers:

- **Flash block:** This add-on will prevent Flash ads from playing until the user opts to allow them.
- **HTTPS Everywhere:** This plugin encrypts a user's web browsing traffic.

- **No Script or Script Safe:** These programs are popular and block scripts on websites unless the user explicitly accepts to run them.

e) Other steps users can take to harden the browser

All of these actions focus on manipulating the browser to be more secure. It is also advisable to add another layer of security such as implementing firewalls, installing antivirus software, and avoiding unsafe behaviour, the Home Network Security Incident document outlines.

## 5. Using Multiple Browsers

A user may install multiple web browsers as a way to improve security. Document viewers and email clients may use another browser or offer various functionalities depending on the browser in use. Specific browsers may be necessary to open certain file types. The point is that one web browser will not necessarily fit all of a user's applications and purposes. It is therefore essential to securely configure each web browser available on a computer. There is a distinctive advantage in dedicating one web browser for sensitive activities such as online banking while dedicating another for general-purpose web browsing. The use of multiple browsers greatly minimizes the probability of compromising sensitive information in a specific browser, website, or software.

Improving internet security for web browsers will protect networked data and computer devices from malware or privacy breaches.

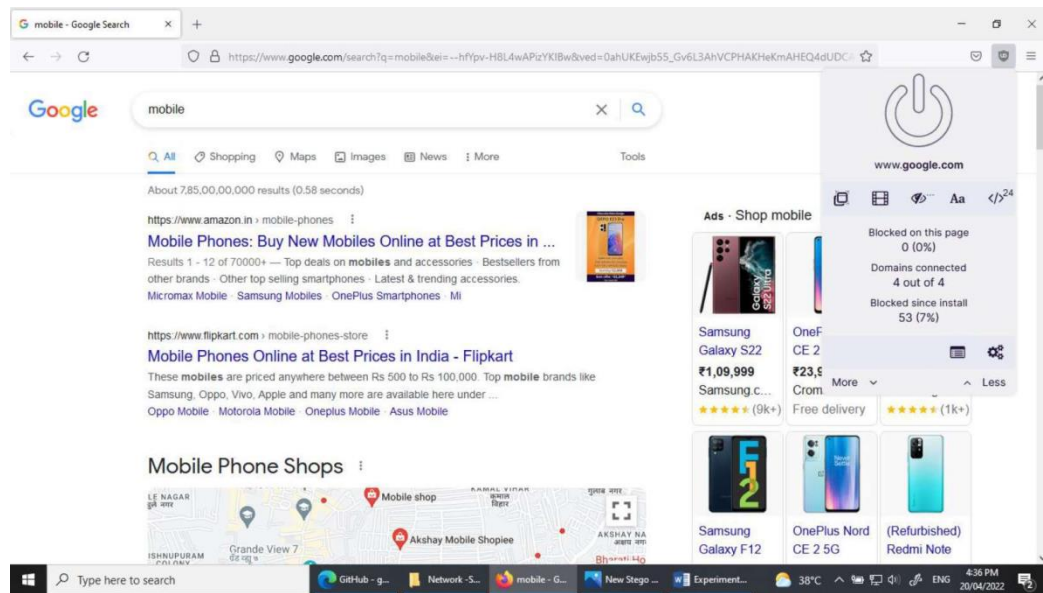
### Procedure:

#### 1) **Install add on uBlock Origin (or uBlocko) to Mozilla browser.**

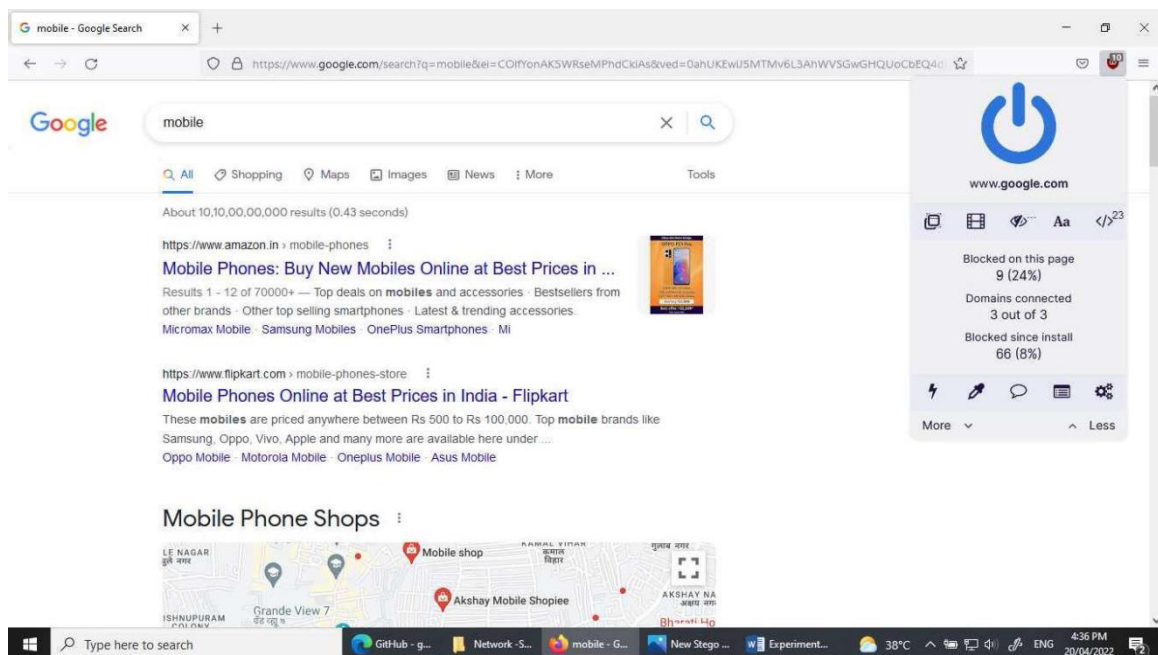
It is not an *ad blocker*; it's a general-purpose blocker. uBlock Origin blocks ads through its support of the Adblock Plus filter syntax. uBlock Origin extends the syntax and is designed to work with custom rules and filters. Furthermore, advanced mode allows uBlock Origin to work in default-deny mode, which mode will cause all 3rd-party network requests to be blocked by default, unless allowed by the user.

<https://addons.mozilla.org/en-US/firefox/addon/ublock-origin/>





**With uBlock origin installed but turned OFF both legitimate and non-legitimate advertisements are visible.**

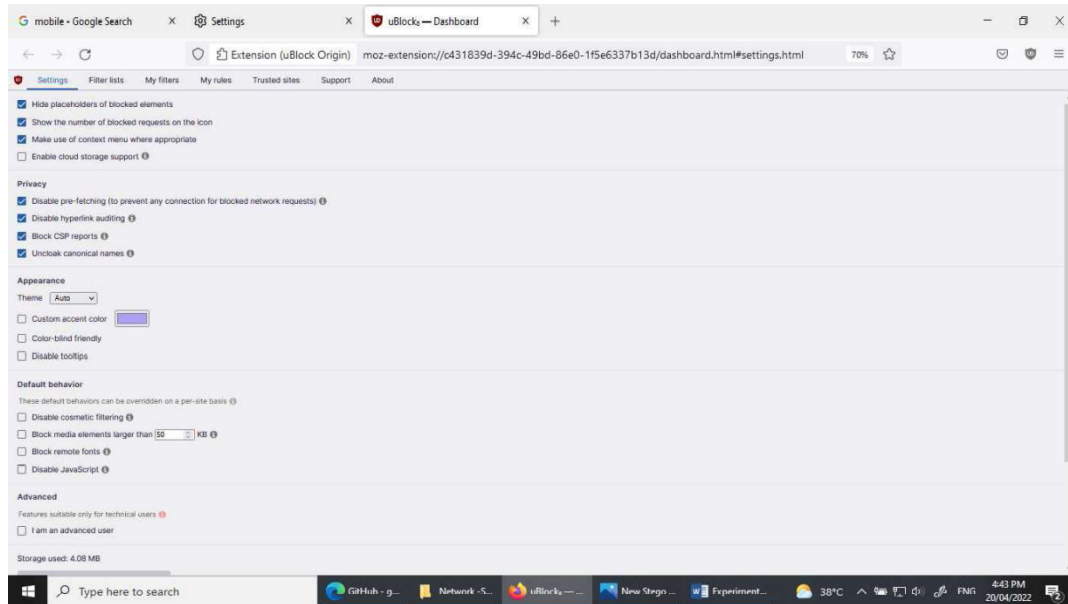


**With uBlock origin installed and turned ON both legitimate and non-legitimate advertisements are blocked.**

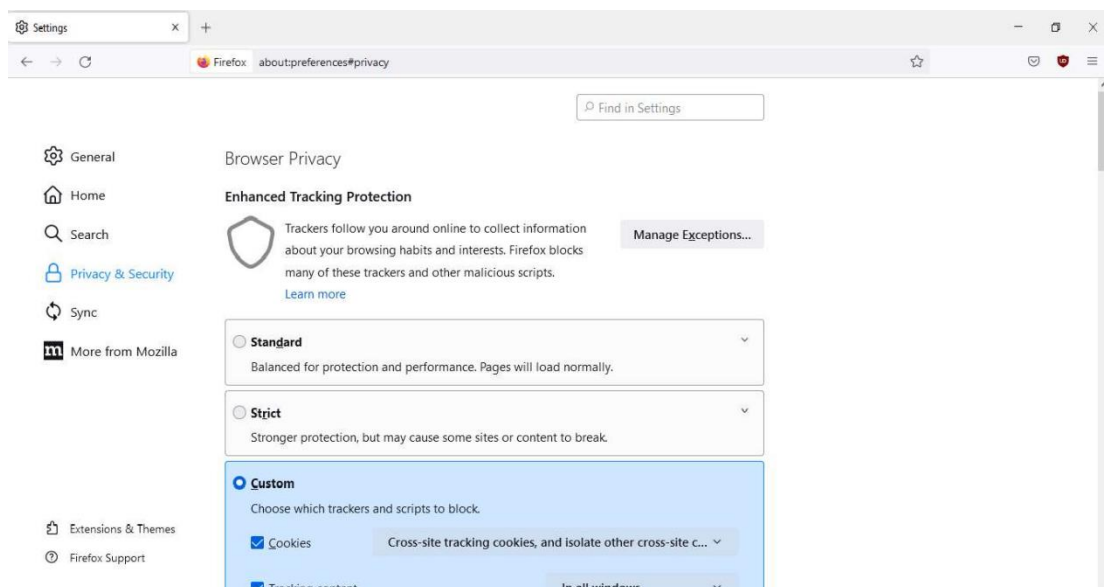
Pop ups are commonly used by threat actors as a means to infect computers with malicious code. The pop up may try to coerce users into accessing unsafe web pages, or downloading malware. There are various techniques for forcing users to interact with the popup—attackers

may create a popup that cannot be closed, or include a warning that will urge the user to download a malicious payload.

- 2) Go To dashboard of uBlock Origin and choose preferences.



- 3) Open Mozilla Firefox Browser, Go to settings, then go to Privacy & Security. [Privacy and security settings | Firefox Help \(mozilla.org\)](https://support.mozilla.org/en-US/kb/privacy-and-security-settings)



- 4) To keep your information safe and secure with Firefox's private browsing, password features and other security settings, configure the following preferences;

**a) HTTPS-Only Mode in Firefox**

b) When you use HTTPS-Only Mode, this ensures all of your connections are encrypted and secure. So you have peace of mind that no one can snoop on the content of the pages you visit or hack into your connection to a website to steal your passwords, credit card information, or other personal information. This is especially useful when you are using a public WiFi where you can't be sure of the integrity of your internet connection.



Some websites only support HTTP and the connection cannot be upgraded. If HTTPS-Only Mode is enabled and a HTTPS version of a site is not available, you will see a “Secure Connection Not Available” page: If you click **Continue to HTTP Site** you accept the risk and then will visit a HTTP version of the site. HTTPS-Only Mode will be turned off temporarily for that site. Click the **Go Back** button if you want to avoid any unencrypted connections.

**b) Does Firefox share my location with websites?**

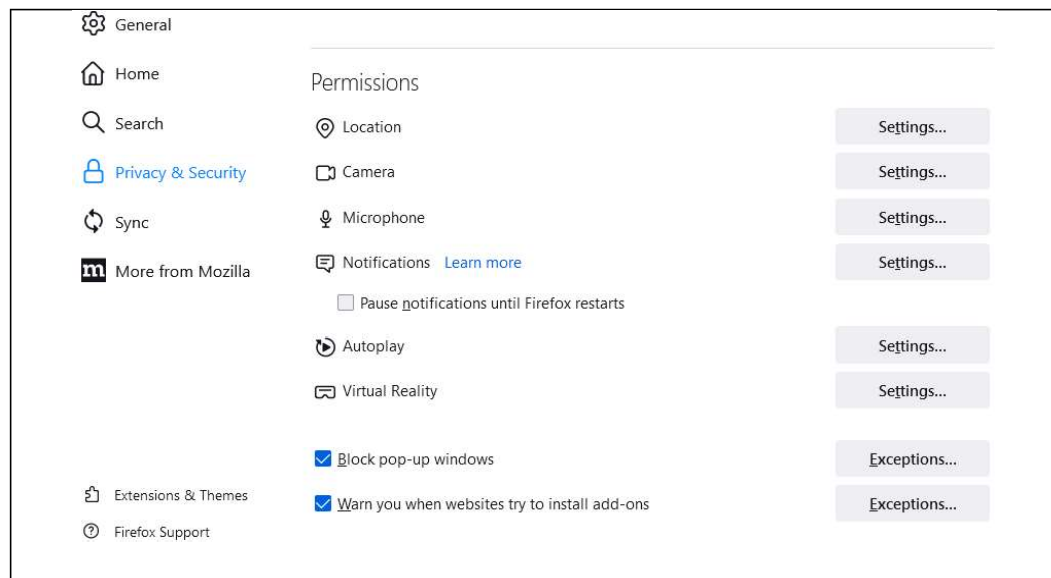
Firefox doesn't share your location without your permission, by default, Firefox uses Google Location Services to determine your location by sending:

- your computer's IP address,
- information about the nearby wireless access points, and



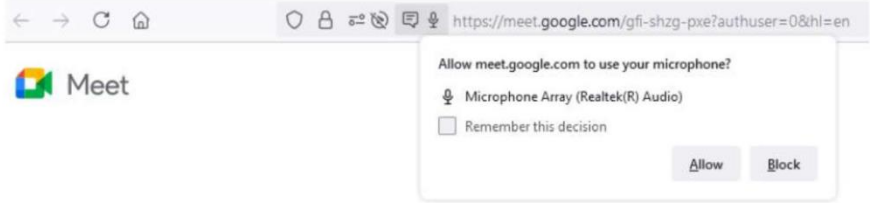
- a random client identifier, which is assigned by Google, that expires every 2 weeks.

Google Location Services then returns your estimated geolocation. The information is exchanged over an encrypted connection to protect your privacy. Once Firefox has your location information, it passes it to the website that requested it. The name and location of the website you are visiting are never shared with Google Location Services.



### c) Manage your camera and microphone permissions with Firefox

1. Open Firefox and go to the desired site.
2. A confirmation pop-up displays asking if you want to share your device's camera and microphone with that site.



The screenshot shows a Firefox browser window with the address bar displaying 'https://meet.google.com/gfi-shzg-pxe?authuser=0&hl=en'. The page content shows the Google Meet logo. A permission pop-up is displayed over the page, asking 'Allow meet.google.com to use your microphone?'. It lists the microphone as 'Microphone Array (Realtek(R) Audio)' and includes a checkbox for 'Remember this decision'. There are 'Allow' and 'Block' buttons at the bottom of the pop-up.

3. Select the ☐ **Remember this decision** checkbox if you want Firefox to remember your selections for this site so that they can be used by default in the future.
4. Click the  button if you want to share these devices with the site.
5. Click the  button if you don't want to share these devices with the site.

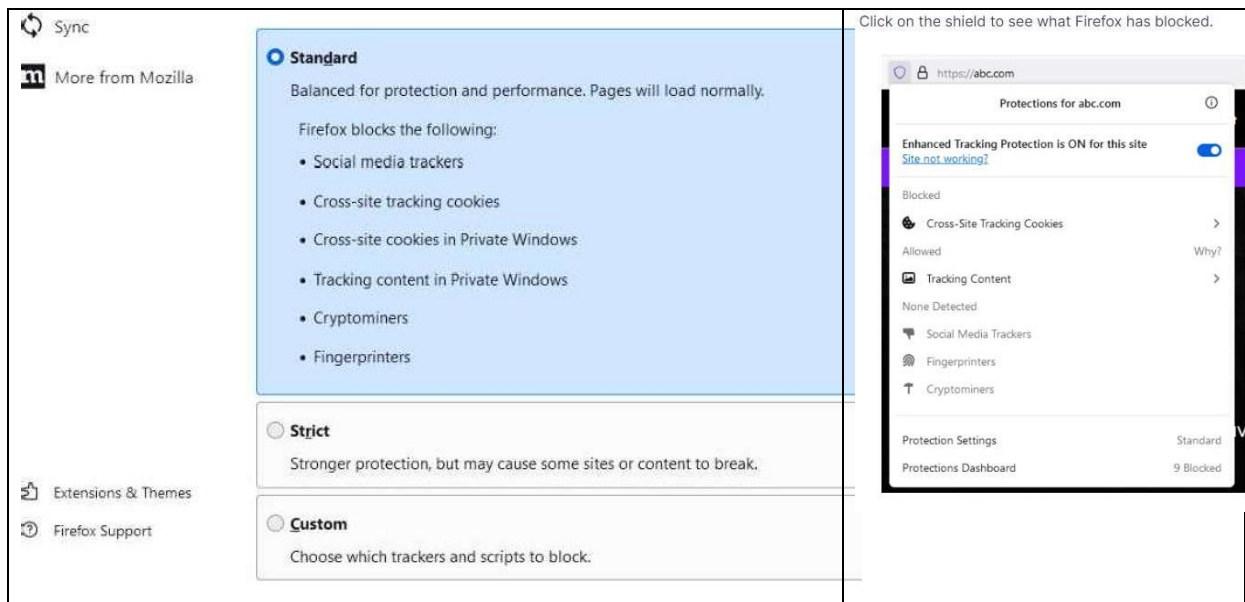
**d) SmartBlock for Enhanced Tracking Protection**

Smart Block stands in for common tracking scripts, which are blocked by private browsing mode and strict tracking protection. By doing so, it lets pages load more fully, with less breakage, without you having to do anything — all while keeping those tracking scripts blocked.

**e) Enhanced Tracking Protection & Total Cookie Protection in Firefox**

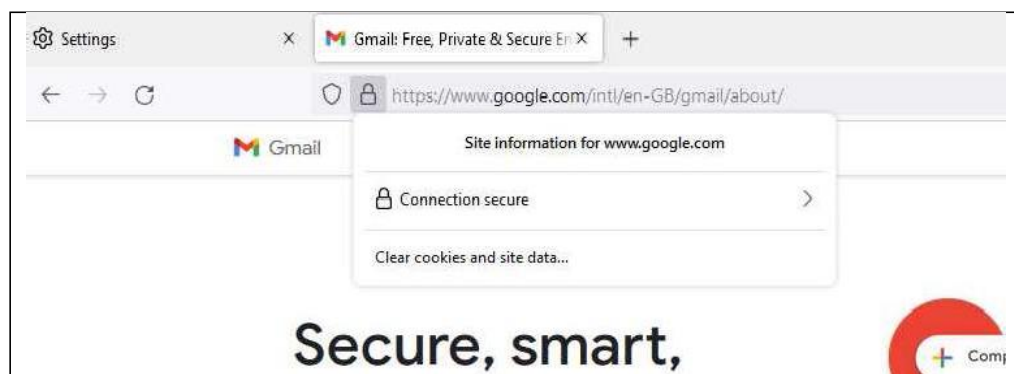
Enhanced Tracking Protection in Firefox automatically protects your privacy while you browse. It blocks cookies & trackers that follow you around online to collect information about your browsing habits and interests without breaking site functionality. It also includes protections against harmful scripts, such as malware that drains your battery. Enabling Total Cookie Protection within Enhanced Tracking Protection will block third-party cookies to prevent tracking companies from tracking your browsing from site to site. However, this may cause some websites not to work correctly, which is referred to as site breakage. By default, Firefox blocks the following types of trackers and scripts.

- Social media trackers
- Cross-site tracking cookies (also isolates remaining cookies)
- Fingerprinters
- Cryptominers
- Tracking content: These trackers are hidden in ads, videos, and other in-page content. In **Standard** mode, tracking content is blocked in Private Windows. To add this protection in all windows, visit your privacy preferences and select **Strict** or **Custom**.



#### f) Site Information panel

The Site Information panel in Firefox tells you about a website's connection security and identity. A padlock icon  in the address bar will indicate whether the connection to the website is fully secure. For more information, click the padlock to open the **Site Information** panel. This panel shows you whether the current connection is classified as secure.



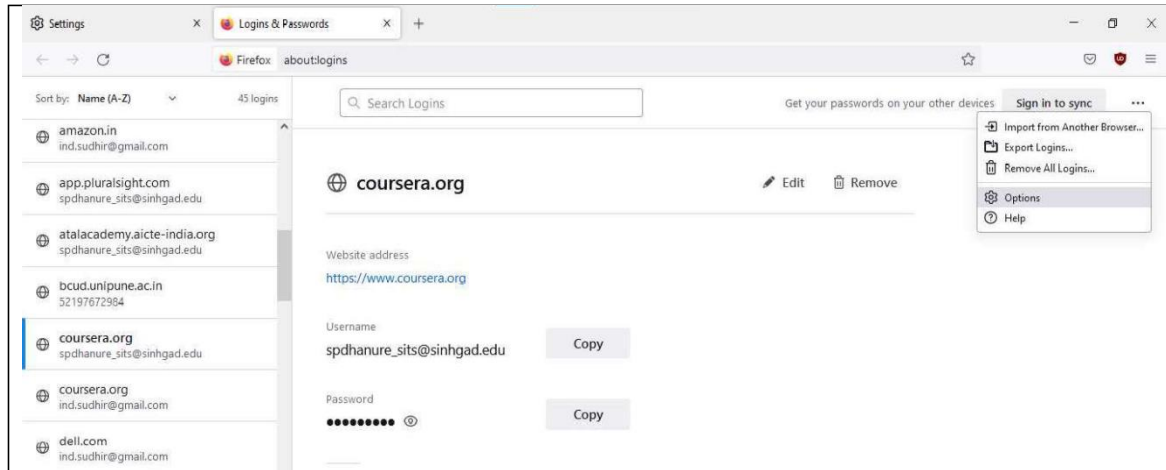
#### g) Autofill logins on Firefox

Firefox lets you control whether or not to autofill your logins and passwords. Disable autosave for specific websites

To prevent Firefox from saving logins and passwords for a certain website:

1. Click on the menu button  to open the menu panel.

2. Click Passwords. The Firefox *about:logins* page will open in a new tab.
3. Click the Firefox logins menu (three dots), then click Options.



## h) Telemetry collection and deletion

Firefox collects telemetry data by default to help improve the performance and stability of Firefox. Telemetry data is made up of two data sets: interaction data and technical data.

### Firefox Data Collection and Use

We strive to provide you with choices and collect only what we need to provide and improve Firefox for everyone. We always ask permission before receiving personal information.

[Privacy Notice](#)

- ☒ Allow Firefox to send technical and interaction data to Mozilla [Learn more](#)
- ☒ Allow Firefox to make personalized extension recommendations [Learn more](#)
- ☒ Allow Firefox to install and run studies [View Firefox studies](#)
- ☐ Allow Firefox to send backlogged crash reports on your behalf [Learn more](#)

To opt out of sending any Firefox telemetry information:

1. Click the menu button and select **Settings**.
2. Select the **Privacy & Security** panel.
3. Scroll to the *Firefox Data Collection and Use* section.
4. Deselect the **Allow Firefox to send technical and interaction data to Mozilla** checkbox.

To learn more about what data Mozilla receives and how it's used, see [Mozilla's Firefox Privacy Notice](#).

**i) Automatically fill in credit card data on Web forms**

Manage credit card autofill settings in Firefox version 81 and above. Firefox lets you automatically fill in your saved credit card information on Web forms (for example, when you're ordering something online). As a precaution, your CVV number is **not** saved. You can also choose to password protect your credit card data for an additional layer of protection.

### Turn credit card autofill on or off

This feature is recommended for private computers **only**.

1. Click the menu button  and select **Settings**.
2. Click the **Privacy & Security** panel and scroll down to the **Forms and Autofill** section.
3. Click the checkbox next to **Autofill credit cards** to turn it on, or uncheck the box to turn it off.

**Advanced: Disable credit card autofill in about:config**

This feature can also be disabled manually by modifying the following preferences via the [Configuration Editor \(about:config\)](#) to **false** and then restarting Firefox:

`extensions.formautofill.creditCards.enabled`  
`extensions.formautofill.creditCards.available`

**Note:** Many other security and privacy settings are available with the browser. [Privacy and security settings | Firefox Help \(mozilla.org\)](#)

**Conclusion:**

---

---

---

---

---